המכללה האקדמית להנדסה בראודה בכרמיאל

פרויקט מסכם מערכות הפעלה

מגישים :

אל קלעאני ניסאן .

סלאח אלדין ענאן.

מוגשת ל: ד"ר רפאל שללה

תקציר:

ניתוח ציונים באמצעות ריבוי תהליכים וריבוי אשכולות

הפרויקט מעבד קבצי ציונים של תלמידים תוך שימוש בריבוי אשכולות (Multi-threading) לקריאת הנתונים וריבוי תהליכים (Multi-processing) לחישובים סטטיסטיים, במטרה לייעל את זמן הריצה ולשפר את הביצועים.

פונקציות עיקריות:

1. קריאת נתונים (באמצעות ריבוי תהליכונים):
   - התוכנית קוראת מספר קבצי טקסט המכילים ציונים של תלמידים במקביל.
   - כל קובץ מייצג תחום לימוד (מקצוע), והקריאה מתבצעת באמצעות תהליכונים כדי לזרז את התהליך.
2. חישובים סטטיסטיים (באמצעות ריבוי תהליכים):
   - לכל מקצוע מחושבים ממוצע, חציון והתפלגות הציונים על פני קטגוריות מוגדרות מראש.
   - התהליך מחולק לכמה תהליכים נפרדים, וכל תהליך מחשב היסטוגרמה על חלק מהנתונים.
   - לאחר עיבוד הנתונים, התוצאות מאוחדות ומחושבת התפלגות האחוזים לכל טווח ציונים.
3. כתיבת דוח סיכום:
   - הנתונים הסטטיסטיים עבור כל מקצוע נכתבים לקובץ סיכום ("grades_summary.txt").
   - הסיכום כולל את שם המקצוע, מספר הציונים הכולל, ממוצע, חציון, והתפלגות הציונים באחוזים.
4. שימוש במנגנוני סנכרון:
   - נעילות (Locks) משמשות למניעת התנגשויות בגישה למשאבים משותפים כמו רשימות ותורים.
   - תור (Queue) משמש להעברת נתונים בין תהליכים בצורה בטוחה ויעילה.

מטרת הפרויקט:

להאיץ את עיבוד הנתונים ולהפוך את חישוב הסטטיסטיקות לאפקטיבי יותר באמצעות שילוב בין ריבוי תהליכונים לקריאה מקבילה וריבוי תהליכים לעיבוד חישובי.

האובייקט המרכזי (Class Subject) :

הגדרנו לכל קובץ של מקצוע אובייקט משלו .

לכל אובייקט קיים

1. שם מקצוע (name)
2. מערך ציונים(scores).
3. כתובת של הקובץ(data path).
4. ציון ממוצע (mean).
5. חציון(median)
6. מסנכרן Lock לסנכרון בין התהליכים .

```python
class Subject:
    def __init__(self, name, scores, data_path):
        self.name = name  # Subject name
        self.scores = scores  # List of scores for the subject
        self.lock_process = multiprocessing.Lock()  # Lock for process synchronization
        self.data_path = data_path  # Path to store output data
        self.mean = None  # Variable to store mean of scores
        self.median = None  # Variable to store median of scores
```

הפרויקט מחולק ל 3 שלבים

שלב 1 :קריאה + שמירה :

קוד הגדרת הכתובת :

הקוד מבצע את הפעולות הבאות :

1. קבלת הכתובת הנוכחית .
2. הגדרת כתובת ה data ובדיקה אם התיקייה קיימת .
3. שינוי הכתובת לכתובת של התיקייה data.

```python
current_directory = os.getcwd()  # Get current working directory
print(f"The current directory is: {current_directory}")
# Define data directory path
data_path = os.path.join(current_directory, "data")

# Check if data directory exists
if not os.path.exists(data_path):
    print(f"Data directory not found at: {data_path}")
    sys.exit(1)  # Exit if data directory does not exist

os.chdir(data_path)  # Change working directory to data folder
print(f"The directory changed to {os.getcwd()}")
```

הגדרת פונקציה לקריאה ושמירה (read_and_save):

הקוד מבצע את הפעולות הבאות:

1. הפונקציה פותחת קובץ מסווים שנמצא בתיקייה data .
2. מבצעת קריאה לכל שורה שמייצגת ערך של ציון בקובץ ושומרת הציון למערך הציונים scores.
   - חשוב להדגיש כי scores הוא מערך ציונים שקיים בכל אובייקט .
3. לאחר סיום קריאת הציונים הפונקציה שומרת אובייקט עם המשתנים המסוימים שלו (שם, מערך ציונים, כתובת הקובץ)

```python
def read_and_save(file_name, data_path):
    global subjects_lists  # Access global subjects list
    scores = []  # List to store scores
    subject_name = file_name.replace(".txt", "")  # Extract subject name from file name

    # Read scores from file
    with open(file_name, "r") as file:
        for line in file:
            scores.append(float(line.strip()))  # Convert score to float and store

    lock_threading.acquire()  # Acquire lock for thread safety
    subjects_lists.append(Subject(subject_name, scores, data_path))  # Append new Subject object to list
    lock_threading.release()  # Release lock
```

- נעשה שימוש בשיטת סנכרון lock_thraeading כדי לשמור נתונים למערך subjects_lists בצורה בטוחה ,בלי לגרום לחפיפה בין התהליכונים.

פתיחת תהליכונים (threading) לקריאה לפונקציה(read_and_save) :

השתמשנו בתהליכונים **(threading)** מכוון שמדובר בפעולת קריאה שלא תצטרך התערבות של המעבד .

הקוד מבצע את הפעולות הבאות

1. מקבל רשימה של כל הקבצים ב תיקייה data .
2. הקוד מוודא אם הקובץ נמצא ותקין.
3. פותח תהליכון עבור כל קובץ שהמשימה שלו היא לקרוא ולשמור את הקובץ באמצעות הפונקציה read_and_save .

```python
threads = []  # List to store threads
files = os.listdir()  # Get list of files in directory
# Create threads for reading data files
for file_name in files:
    if file_name.endswith(".txt") and os.path.isfile(file_name):  # Ensure it's a valid file
        print(f"Reading file: {file_name}")
        thread = threading.Thread(target=read_and_save, args=(file_name, data_path))
        threads.append(thread)
        thread.start()

for thread in threads:
    thread.join()  # Wait for all threads to finish
```

*הקוד מחכה לסיום הקריאה של כל התהליכונים באמצעות הפעולה join כדי לוודא שכל הקבצים נקראו ונשמרו לפני המעבר לחישובים .

שלב 2 :חישובים :

פונקציה 1 :(calculate_stats)

פונקציה מקומית בתוך האובייקט שמחשבת את הממוצע החציון והתפלגות הציונים.

לפי ההנחיות בפרויקט חישובי הממוצע ,החציון וגם הכתיבה לקובץ לא מצטרך שימוש ב תהליכים כי הוא חישוב לא כבד על המעבד .

הפונקציה מקבלת מספר התהליכים שצריך לייצר כדי לחשב את התפלגות הציונים.

מספר התהליכים הוא מספר הליבות במחשב מקבלים את המספר הזה כדי לחלק את כובד החישוב של ההתפלגות על כל הליבות

הפונקציה מבצעת את הפעולות הבאות .

```python
def calculate_stats(self, num_of_process, queue):
    if not self.scores:  # Check if there are scores available
        print(f"Warning: No scores found for {self.name}")
        return

    self.mean = np.mean(self.scores)  # Calculate mean of scores
    self.median = np.median(self.scores)  # Calculate median of scores
    self.process_subjects_in_parallel(num_of_process, queue)  # Start parallel processing

    histogram_all = np.zeros(5)  # Initialize histogram array
    while not queue.empty():  # Retrieve histogram results from queue
        histogram_all += np.array(queue.get())

    total_scores = len(self.scores)  # Get total number of scores
    percentage_scores = (histogram_all / total_scores) * 100  # Convert histogram to percentage
    self.write_data(percentage_scores)  # Write results to file
```

1. חישוב ממוצע + חציון ללא שימוש בתהליכים בעזרת פונקציית numpy כדי להבטיח יעילות .
2. קריאה לפונקציה process_subjects_in_parallel לחישוב התפלגות הציונים(Histogram) בשימוש בתהליכים מקבליים(process) .
3. אחרי סיום התהליכים הטור queue מקבל לתוכו מערך התפלגויות בגודל מספר הקורים במחשב , שכל התפלגות כבר חושבה על ידי כל תהליך בנפרד.
4. חישוב אחוזי ההתפלגות .
5. כתיבה לקובץ על ידי כריאה לפונקציה Write_data_.

הפונקציה (process_subjects_in_parallel) :

הפונקציה מבצע את הפעולות הבאות:

```
def process_subjects_in_parallel(self, num_of_process, queue):
    processes = []  # List to store process instances
    chunk_size = len(self.scores) // num_of_process  # Determine chunk size for each process

    # Creating and starting multiple processes for histogram calculation
    for proc in range(num_of_process):
        start_idx = proc * chunk_size  # Start index for chunk
        end_idx = len(self.scores) if proc + 1 == num_of_process else (proc + 1) * chunk_size  # End index for chunk
        p = multiprocessing.Process(target=self.histogram, args=(self.scores[start_idx:end_idx], queue))  # Creating process
        processes.append(p)  # Adding process to list
        p.start()  # Starting process

    # Waiting for all processes to complete
    for p in processes:
        p.join()
```

1. מפצלת את מערך ה scores למערכים לפי מספר הליבות של המעבד .
2. עבור כל מקטע של ציונים מקדישים ליבה לביצוע העבירה על הציונים ומיון לפי תחומים בעזרת הפונקציה . histogram
3. *הקוד מחכה לסיום הקריאה של כל התהליכים באמצעות הפעולה join כדי לוודא שכל הקבצים נקראו ונשמרו לפני המעבר לכתיבה לקובץ.

*השימוש בתהליכים נועד לנצל את רוב יכולת המעבד על ידי הפעלת כל הקורים לביצוע פעולות חישוביות .

*חשוב להדגיש כי הקוד אינו יעיל מבחינת זמן עבור קבצים קטנים, מכוון שהקוד מבזבז זמן לחלק המערך לתהליכים ולחכות עד שייסגרו יותר מאשר לבצע חישובים .

בחרנו להמשיך עם הקוד הזה כדי להצטמד לתאוריה ולנצל את כל יכולת המעבד עבור חישובים כבדים .

הפונקציה (histogram):

```
def histogram(self, scores_list, queue):
    # Create a histogram of scores with predefined bins
    histogram, bin_edges = np.histogram(scores_list, bins=[0, 40, 55, 70, 85, 101])
    queue.put(histogram)  # Store histogram in queue
```

1. מחשבת ומחזירה מערך של התפלגות הציונים עבור מערך מסוים שמשויך עבור כל תהליך .
2. שומרת המערך לטור queue כדי לחשב היסטוגרמה כוללת עבור כל הציונים בתוך הפונקציה calculate_stats.

- השימוש ב queue מבטיח סנכרון בין התהליכים בלי שימוש בשיטת סנכרון (lock/semaphore).

שלב 3 :כתיבה ושמירה לקובץ :

1. פתיחת קובץ grades_summary וכתיבה לתוכו את הפרמטרים של האובייקט (שם , מספר ציונים, ממוצע, חציון, היסטוגרמה) לפי הדרישות .

```python
def write_data(self, percentage_scores):
    total_grades = len(self.scores)  # Total number of grades
    print(f"Subject: {self.name}")
    print(f"Mean: {self.mean}")
    print(f"Median: {self.median}")

    # Define summary file path
    summary_file_path = os.path.join(self.data_path, "grades_summary.txt")

    # Prepare summary content
    main_content = f"""
        Domain: {self.name}
        Total Grades: {total_grades}
        Mean: {round(self.mean, 2)}
        Median: {round(self.median, 2)}
        Histogram:
        0-39: {round(percentage_scores[0], 2)}%
        40-54: {round(percentage_scores[1], 2)}%
        55-69: {round(percentage_scores[2], 2)}%
        70-84: {round(percentage_scores[3], 2)}%
        85-100: {round(percentage_scores[4], 2)}%
          """

    self.lock_process.acquire()  # Lock process to ensure safe file writing
    with open(summary_file_path, "a") as file:
        file.write(main_content)  # Write summary to file
    self.lock_process.release()  # Release lock
    print(f"Summary written for subject: {self.name}")
```

- השימוש ב lock במקרה הזה לא קריטי מכיוון שמדובר בתהליך אחד שממשיך לרשום את הערכים לתוך הקובץ אבל יותר בטוחה.

מעבר על כלל האוביקטים בטור הגלובלי שנשמר על ידי התהליכונים :

1. קריאה לפונקציה המבצעת חישובים עבור כל אובייקט .
   - על כל אוביקט מופעלים כל ליבות המחשב משימיים החישובים ועוברים לאובייקט שאחרי .
   - השיטה הזו יעילה עבור קבצים גדולים שמצריכים כוח חישובי רחב ומבטיחה השימוש בכל הליבות .

```python
# Process subjects using multiprocessing
print("\nStarting multiprocessing to calculate stats...")
for subject in subjects_lists:
    subject.calculate_stats(num_cores, queue)

print("\nAll statistics calculated and written to file .")
```

-
- דוגמה לפלט:

```
Domain: Math
Total Grades: 10000
Mean: 67.76
Median: 69.0
Histogram:
0-39: 12.07%
40-54: 16.19%
55-69: 22.54%
70-84: 22.09%
85-100: 27.11%

Domain: Science
Total Grades: 9000
Mean: 73.33
Median: 74.0
Histogram:
0-39: 4.01%
40-54: 12.29%
55-69: 24.38%
70-84: 28.51%
85-100: 30.81%
```